



PhishGuard 프론트엔드(C 트랙) 구현 정리

feat/front-c

작성일 2026-07-04 · 개발가이드 C1~C8 + 신고기능 상세스펙 + 유저플로우 문서 대비 현황

1. 개요

개발가이드.pdf 의 C 트랙(프론트·데모) 명세, "신고 기능(Report) 상세 스펙" 섹션, 그리고 이후 별도로 작성된 유저플로우 mermaid 문서까지 세 문서를 기준으로 지금까지 구현한 내용을 정리합니다. 세 문서 사이에 표현이 다른 지점이 하나 있어 5번 항목에 별도로 표시해두었습니다.

2. C1~C8 체크리스트 대비 구현 현황

단계	목표	상태	구현 위치	비고
C1	Vite React 세팅 + 백엔드 프록시	완료	vite.config.js	/api → localhost:8000 프록시 설정됨
C2	API 클라이언트 (계약 ①②③)	완료	src/api/client.js	계약대로 구현 + res.ok 체크/에러 throw 보장. getGraph / getTrends 는 normalize.js 를 거쳐 표준 형태로 반환
C3	입력 화면(textarea + 발신번호 + 버튼)	완료	src/pages/Analyze.jsx	
C4	결과 화면(계약 ① 필드 전부)	완료	src/components/Result.jsx	risk_score/level/reasons/evidence/signals/cluster 전부 매핑. 로딩 스켈레톤·에러·빈 상태·카운트업 애니메이션 포함
C5	그래프 시각화 (ForceGraph2D)	완료	src/components/OrgGraph.jsx , src/pages/Graph.jsx	edges→links 매핑, nodeLabel 포맷 가이드와 동일. 유저플로우 문서 반영해 독립된 화면(/graph)으로 분리 + 수동 새로고침 버튼 추가
C6	신고 버튼 → 신고 페이지	완료	src/pages/Report.jsx	라우팅/프리필/제출/문구 템플릿까지 완료. 제출 후 동작은 5번 항목 참고
C7	트렌드 대시보드	완료	src/pages/Trends.jsx	최다 신고 문구/URL/사칭기관 3개 랭킹 차트. 에러·빈 데이터 안내 상태 포함
C8	데모 시나리오 + 발표덱	미착수	—	엔지니어링 작업이 아니라 발표 준비 영역이라 범위 밖으로 둠

3. 신고 기능(Report) 상세 스펙 대비

항목	상태	비고
AI 자동 판단은 DB 저장 안 함	완료	report() 는 사용자가 버튼을 눌렀을 때만 호출됨. 자동 저장 경로 없음
buildReportText() 신고 문구 자동생성	완료	src/utlis/buildReportText.js — 가이드의 신고 문구 템플릿과 섹션 순서 1:1 일치
신고 섹션 UI: 문구 미리보기 + 버튼	완료	미리보기는 textarea로 수정 가능. 버튼은 원래 표 기준 2개(PhishGuard 신고 / KISA 이동)로 확정 — "통합신고센터" 버튼은 논의 후 제외

PhishGuard 신고 버튼 동작	확인 필요	5번 "열린 결정 사항" 참고
AnalyzeResponse.urls 필드 대응	완료(방어적 처리)	B(백엔드) 작업 전까지는 evidence 의 type: 'url' 항목으로 폴백. 필드 추가되면 자동으로 우선 사용됨

4. 유저플로우 문서 반영 — 화면 구조 재편

이후 전달된 유저플로우 mermaid 문서를 기준으로 화면 구성을 다음과 같이 다시 짚습니다.

라우트	화면	변경 내용
/	Home (신규)	랜딩 허브. "문자 분석 시작 / 조직 그래프 / 위협 트렌드" 3갈래 진입 카드
/analyze	문자 분석	기존 홈 역할을 분리. 그래프·번호위젯을 빼서 입력→결과 흐름만 남김. 결과 화면에 "다시 분석하기" 버튼 추가
/graph	조직 그래프 (신규)	기존엔 다른 화면에 끼워져 있던 그래프를 독립 화면으로 분리, 수동 새로고침 버튼 추가
/report	스미싱 신고	기존과 동일 (문구 생성/수정/제출)
/trends	위협 트렌드	데이터 로드 실패 시 안내 배너, 카테고리별 빈 데이터 안내 문구 추가

유저플로우 문서에 없던 "번호·URL 조화", "스미싱 예방 가이드" 화면은 별도 제안 기능이었으나, 문서와 정확히 일치시키기 위해 삭제했습니다. (라우트·메뉴·파일 전부 제거)

5. 열린 결정 사항 (Open Decision)

확인 필요 · 추후 변경 가능

PhishGuard 신고 버튼을 누른 뒤 화면이 어떻게 되어야 하는가?

- 가이드 PDF (신고 기능 상세 스펙, 버튼 3개 정리 표): PhishGuard 신고 버튼의 "이동/대상"이 (내부, 이동 없음) 로 명시됨 — 즉 버튼을 눌러도 신고 페이지에 그대로 머물러야 함
- 유저플로우 mermaid 문서: n21(제출)→n22(완료 메시지)→n23(그래프 갱신)→n24(조직 그래프 화면)로 이어지는 흐름이 그려져 있음 — 즉 제출 후 조직 그래프 화면으로 이동해야 함

두 문서가 서로 다른 동작을 요구하고 있어, 현재는 유저플로우 문서 기준(그래프 화면으로 이동)으로 구현되어 있습니다. 다만 이 결정은 아직 확정된 것이 아니라는 걸 명시하기 위해 이 섹션에 남겨둡니다.

변경은 src/pages/Report.jsx 의 handleReportToDb 함수 안 navigate('/graph', { state: { justReported: true } }) 한 줄만 고치면 되도록 구조화해줬습니다. (예: 이동을 없애고 싶으면 이 줄을 지우고, 성공 메시지를 신고 페이지 자체에 인라인으로 표시하는 코드로 교체)

6. 통합 레이어 — 백엔드 연결 시 안전장치

src/api/client.js 와 src/api/normalize.js 가 프론트-백엔드 경계를 담당합니다. 실제 백엔드가 계약과 다른 필드명으로 응답을 주더라도, 이 두 파일만 고치면 나머지 화면 코드는 전혀 손댈 필요가 없도록 설계했습니다.

- getGraph() — 응답을 {nodes, edges, cluster_count} 표준 형태로 정규화 (예: links / clusterCount 로 와도 흡수)
- getTrends() — /api/trends 스키마가 아직 미확정이라, 후보 필드명(label/phrase/url/domain/org , count/report_count/reports)을 넓게 받아 {label, count} 로 통일

- `analyze()` / `report()` — 계약이 고정된 요청 형식이라 별도 정규화 없이 그대로 전달

7. 파일 구조

frontend/src/	
├ App.jsx	라우트 정의 (<code>/</code> , <code>/analyze</code> , <code>/graph</code> , <code>/report</code> , <code>/trends</code>)
├ main.jsx	
├ index.css	전역 스타일 + 커스텀 애니메이션 키프레임
├ api/	
│ └ client.js	<code>analyze</code> / <code>getGraph</code> / <code>getTrends</code> / <code>report</code>
│ └ normalize.js	백엔드 응답 → 표준 형태 정규화
├ components/	
│ └ AppShell.jsx	헤더 + 하단 탭바 + 우측 슬라이드 메뉴
│ └ Logo.jsx	커스텀 브랜드 로고 (방패+체크+스파클)
│ └ OrgGraph.jsx	ForceGraph2D 그래프 렌더링
│ └ RankedBarList.jsx	랭킹 막대 리스트 (트렌드/번호 랭킹 공용)
│ └ Result.jsx	분석 결과 화면
│ └ Splash.jsx	시작 스플래시 애니메이션
├ pages/	
│ └ Home.jsx	랜딩 허브 (신규)
│ └ Analyze.jsx	문자 분석 입력+결과
│ └ Graph.jsx	조직 그래프 화면 (신규)
│ └ Report.jsx	스미싱 신고
│ └ Trends.jsx	위협 트렌드 대시보드
└ utils/	
│ └ buildReportText.js	신고 문구 템플릿 생성
│ └ graphInsights.js	그래프에서 번호 랭킹 추출
│ └ mockGraph.js	백엔드 미준비 시 폴백 mock 데이터